

Paper 02 - Correction Surface

CQE-paper-02

CQECMPLX Formal Paper Series

Review-facing PDF built 2026-06-12

Construction Basis

Treat failure, mismatch, and nonlinear residue as positive correction data rather than dismissal.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- formal paper: production\formal-papers\CQE-paper-02\FORMAL_PAPER.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-02\paper_kernel_manifest.json
- verifier: production\formal-papers\CQE-paper-02\verify_correction_surface.py
- receipt: production\formal-papers\CQE-paper-02\correction_surface_receipt.json (Correction surface decomposition: pass)
- body: production\papers\CQE-paper-02\PAPER-BODY.md
- source: production\papers\CQE-paper-02\SOURCE.md
- formal: production\papers\CQE-paper-02\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-02\02-CQE-tool\TOOL.md
- tool_runner: production\papers\CQE-paper-02\02-CQE-tool\run.py
- workbook: production\papers\CQE-paper-02\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-02.md

Paper 02 - Correction Surface

Abstract

Paper 02 formalizes the first repair layer after the LCR carrier. Once a local state is represented as (L, C, R) , the system needs a disciplined way to handle mismatch. This paper defines a correction surface as a typed residue map that records where a simpler linear carrier fails to reproduce the target transition. The point is not to call every failure a proof. The point is to prevent failure from being erased.

For the Rule 30 / Rule 90 example already present in the codebase, the correction surface is finite and exact:

```
Rule30(L, C, R) = Rule90(L, R) xor correction(L, C, R)
correction(L, C, R) = C and not R
```

The correction fires exactly on the two LCR states $(0,1,0)$ and $(1,1,0)$. This supplies the paper's concrete theorem and its hand-checkable workbook method.

Proof/Exposure Hierarchy

The proof-carrying content of this paper is the mathematics: the definitions, lemmas, constructions, examples, and receipts that establish the claimed result. Paper 00, hand routes, analog tools, workbook language, and obligation ledgers are supplemental validation and exposure layers. They exist to make the math inspectable, reproducible, and accessible without requiring a particular software stack. The hand route is not the purpose of the paper; it is a way to expose the same state transitions with ordinary marks, tokens, lines, or any equivalent physical substitute.

Role in the Suite

Paper 01 establishes the minimal local carrier. Paper 02 installs proofreading: when a proposed transport does not close, the nonclosing part is stored as a residue with a type, location, and next-step obligation. Later papers can repair, curve, lift, or ledger that residue because Paper 02 prevents it from being discarded.

Definitions

Let $A = \{0,1\}$ and let an LCR state be

```
s = (L, C, R) in A^3.
```

Define the Rule 30 local update:

```
R30(L, C, R) = L xor (C or R).
```

Define the Rule 90 local update:

```
R90(L, R) = L xor R.
```

Define the correction map:

```
corr(L, C, R) = C and not R.
```

A **correction surface** is the set of local states where corr is nonzero, together with the receipt that records how that residue is fed to the next transport step.

Main Claim

Theorem 2.1, Correction Decomposition. For every binary LCR state,

$$R30(L, C, R) = R90(L, R) \text{ xor } \text{corr}(L, C, R).$$

Moreover, `corr` is nonzero exactly on

$$\{(0,1,0), (1,1,0)\}.$$

Proof

Over Boolean values,

$$C \text{ or } R = C \text{ xor } R \text{ xor } (C \text{ and } R).$$

Therefore

$$\begin{aligned} R30(L, C, R) &= L \text{ xor } (C \text{ or } R) \\ &= L \text{ xor } C \text{ xor } R \text{ xor } (C \text{ and } R) \\ &= (L \text{ xor } R) \text{ xor } (C \text{ xor } (C \text{ and } R)) \\ &= R90(L, R) \text{ xor } (C \text{ and not } R). \end{aligned}$$

The last equality holds because `C xor (C and R)` is `1` exactly when `C=1` and `R=0`, and is `0` otherwise. Thus the correction is `C and not R`.

Enumerating the eight LCR states, `C=1` and `R=0` occurs only for `(0,1,0)` and `(1,1,0)`. QED.

Correction Is Not Permission to Overclaim

Paper 02 has a narrow but important epistemic rule:

failed transport -> typed residue -> obligation

It does not say:

failed transport -> accepted theorem

This distinction matters for the whole suite. A mismatch is positive data only because it is preserved with enough structure to be tested later. If the residue is not typed, localized, and replayable, it remains an unresolved failure.

D4 Chart Projection

Existing lattice-forge material labels the eight LCR chart states by D4 axis/sheet coordinates.

Under that codec, the correction firing states are:

(0,1,0) -> axis 2, sheet 0
(1,1,0) -> axis 3, sheet 1

Thus the correction tape can be read as a one-bit projection of the D4 chart state:

$$\text{corr} \neq 0 \text{ iff } (\text{axis}, \text{sheet}) \text{ in } \{(2,0), (3,1)\}.$$

Paper 02 does not require the later D4 or triality claims to be proven here. It only records the projection that later papers can consume.

Hand Reconstruction

- Draw the three LCR cells.
- Fill each cell with a binary token.

- Compute ``Rule30 = L xor (C or R)``.
- Compute ``Rule90 = L xor R``.
- Compute ``corr = C and not R``.
- Verify that ``Rule30 = Rule90 xor corr``.
- If ``corr = 1``, mark a black residue token.
- Copy the residue into an obligation row rather than erasing it.

The supplemental hand exposure should produce two firing states and six non-firing states. The firing states are ``(0,1,0)`` and ``(1,1,0)``.

Code Reconstruction

The production verifier for this polish pass is:

```
production/formal-papers/CQE-paper-02/verify_correction_surface.py
```

It verifies:

1. The Rule 30 / Rule 90 / correction identity on all eight LCR states.
2. The exact correction firing set.
3. The D4 axis/sheet projection for the firing states.
4. The residue ledger shape required by the paper.
5. A falsifier: residue is not automatically accepted as proof.

Validation and Hidden-Guess Layer

Paper 02 diagnostics should ask for a prediction before revealing the correction set:

```
which LCR states make C and not R fire?
Does a nonzero correction prove the original route?
What must be recorded before residue can be used by the next paper?
```

The expected answers are:

```
firing states: (0,1,0), (1,1,0)
proof status: no, correction creates an obligation
required record: state, rule, residue value, source route, next obligation
```

Open Obligations

- Wire this verifier into the installable ``cqe_engine.formal`` interface.
- Reconcile the D4 axis/sheet labels with Paper 03's triality presentation.
- Extend the receipt format so later papers can consume correction rows directly.
- Add peer-review citations for Rule 30, Rule 90, Boolean normal forms, and cellular automaton linearization.

Conclusion

Paper 02 converts mismatch into structured work. The proof is finite: the correction surface is exactly ``C and not R``, and it fires on two of the eight binary LCR states. The scientific discipline is also finite: the residue is useful only when it is preserved as a replayable obligation. This is why correction becomes a surface rather than a trash bin.